TÉCNICO LISBOA

Robotics

MEEC

Advanced Driver Assistance Systems: Simulating Lane Tracing Assist

---

Authors:

Vladimiro José de Medeiros Roque (98589) vladimiro.roque@tecnico.ulisboa.pt
José Pedro da Cunha Rodrigues (113234) jose.p.rodrigues@tecnico.ulisboa.pt
Miguel Rodrigues Ferreira (113289) miguel.r.ferreira@tecnico.ulisboa.pt
Pedro Alexandre Ferreira Dias Lopes (103194) pedroafdlopes@tecnico.ulisboa.pt

Group 9

2024/2025 – 1º Semester, P2

Contents

1 Group Members Contribution

- **Task 1:** José Rodrigues
- **Task 2:** Miguel Ferreira
- **Task 3:** Vladimiro Roque
- **Task 4:** Miguel Ferreira
- **Task 5:** José Rodrigues
- **Task 6:** Pedro Lopes
- **Task 7:** Vladimiro Roque

2 Introduction

This report presents the activities carried out during the second laboratory of the Robotics course. The main objective of this lab was to apply the concepts of kinematics, dynamics, and control studied in theoretical classes to the development and simulation of a Lane-Tracking Assist (LTA) system, a functionality commonly found in modern vehicles.

The simulation focused on a scenario where a car is driven on a highway with lanes clearly marked by white lines. The goal of the LTA system is to automatically correct the vehicle's steering, ensuring it stays within the designated lane. This system is activated whenever the vehicle is about to cross the boundary lines, adjusting the steering to prevent lane departure. The implementation involved fundamental steps, including modeling the car, developing controls for the LTA system, and demonstrating its effectiveness through simulations.

The results obtained are discussed in this report, with detailed explanations of the techniques employed, data analysis, and evaluation of the proposed system's performance.

3 Tasks

3.1 Task 1 - Development of a Joystick-like Control Function

To address Task 1, a function was implemented to simulate the manual steering control of a vehicle using either a gamepad or keyboard inputs. The function leverages the Pygame library to capture input from the user and translate it into steering actions, which are essential for controlling the simulated vehicle.

Implementation Details

- **Keyboard Input:**
  - Continuous input from the left and right arrow keys (`K_LEFT` and `K_RIGHT`) is captured to determine the steering direction.
  - Pressing the left arrow steers the car fully to the left (steering value: `-1`), while pressing the right arrow steers it fully to the right (steering value: `1`). If no key is pressed, the steering remains neutral (steering value: `0`).
- **Gamepad Input:**
  - When a gamepad is connected, the function reads the horizontal axis (`Axis 0`) of the joystick to determine the steering direction and intensity.
  - The joystick input produces a continuous range of values, reflecting the precise position of the joystick. A dead zone filter (`> 0.1`) is applied to ignore minor unintentional movements.
  - The returned steering value directly corresponds to the joystick's horizontal axis position, ranging from `-1` (full left) to `1` (full right).
- **Output:**
  - For keyboard input, the function outputs discrete values:
    - `-1` for left,
    - `1` for right,
    - `0` for neutral.
  - For gamepad input, the function outputs continuous values corresponding to the joystick's position.

Purpose and Functionality

This implementation provides intuitive and responsive steering control for the simulated vehicle. By combining discrete keyboard inputs with the precision of gamepad analog inputs, the system offers flexibility and adaptability for different control preferences, enabling users to effectively interact with the simulation environment.

3.2 Task 2 - Car Model

The kinematic model of the car robot, Figure 1 illustrates a visual representation of the car robot.

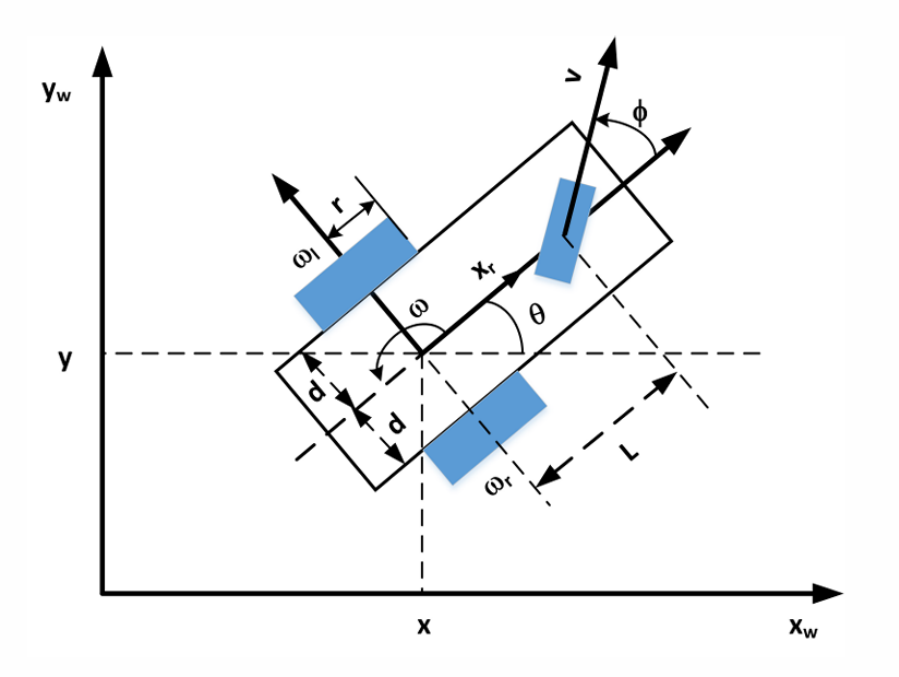


Figure 1: Kinematic model of the car robot

The kinematic model of the car robot, shown in Figure 1, describes the robot's motion in the xOy plane. The velocity components along the x and y axes can be expressed as:

$$\dot{x} = v_r \cos(\theta)$$
$$\dot{y} = v_r \sin(\theta)$$

Where v_r is the robot's linear velocity and θ is its orientation angle. These equations assume no wheel slip and that the robot operates in the plane without roll or pitch.

The angular velocity $\dot{\theta}$ is related to the robot's linear velocity by:

$$\dot{\theta} = \omega = \frac{v_r}{R}$$

Where R is the distance from the reference point to the center of rotation given by:

$$R = \frac{L}{\tan(\phi)}$$

Thus, the rate of change of the orientation is:

$$\dot{\theta} = \frac{v_r \sin(\phi)}{L}$$

The velocity component of the steering is modeled as:

$$v_r = V\cos(\phi)$$

Where V is the linear velocity of the robot and ϕ is the steering angle. Combining these equations, we obtain the following system describing the robot's motion:

$$\dot{x} = V\cos(\theta)\cos(\phi)$$
$$\dot{y} = V\sin(\theta)\cos(\phi)$$
$$\dot{\theta} = \frac{V\sin(\phi)}{L}$$

The steering angle dynamics are given by:

$$\dot{\phi} = \omega_s$$

Where ω_s is the angular velocity of the steering wheel. This model provides a simplified yet effective representation of the robot's motion.

The kinematic model for a car robot can be represented using the following state-space equations:

$$\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\\ \dot{\phi}\end{bmatrix} = \begin{bmatrix}\cos(\theta)\cos(\phi) & 0\\ \sin(\theta)\cos(\phi) & 0\\ \frac{\sin(\phi)}{L} & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}V\\ \omega_s\end{bmatrix}$$

To better approximate the effects of a real car, we limited ϕ to the interval:

$$\phi \in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right]$$

This ensures that the steering angle remains within realistic physical constraints.

Furthermore, to simulate a more accurate vehicle behavior, we incorporated a realistic steering recentralization effect based on both velocity and the caster effect. The caster effect represents the restoring force that tends to return the steering to its neutral position, which is a characteristic of real steering systems.

$$F_{\text{restoring}} = -\text{caster\_effect} \times \phi \times \text{speed\_factor}$$

where:

- ϕ is the current steering angle of the vehicle,
- caster_effect is a constant representing the intensity of the caster restoring effect,
- speed_factor $= \frac{|V|}{\text{wheelbase}+1e-5}$ is the factor that adjusts the force based on the velocity V and the wheelbase.

As the vehicle's speed increases, the restoring force becomes stronger, which naturally encourages the steering to return to the center. This dynamic adjustment of the steering angle not only improves the stability of the vehicle but also ensures more realistic simulation of vehicle dynamics in various driving conditions With this implementation, when the vehicle makes a turn

and the user stops pressing the directional keys, the steering wheel automatically returns to its initial position. This restoring force prevents the vehicle from continuously circling, ensuring that the car stabilizes after a turn and does not keep rotating indefinitely. The system allows for smoother transitions and more intuitive control, simulating a natural steering response that reflects real-world behavior.

3.2.1 Dynamic model for a Car Robot

Generally, the kinematics car model is suitable for trajectory planning at lower speed, however at high-speed scenarios, this prediction model will become unreliable, due to the rise of tire sideslip angles. Therefore, it is necessary to investigate the vehicle dynamics and tire sideslip characteristics for improving the controller performance. A dynamic model incorporates the forces and torques acting on the robot, providing a more accurate representation of its motion.

For this lab, we used the kinematic model due to its simplicity. However, exploring dynamics models is critical for applications like LTA system.

3.3 Task 3: Manual Control of the Car Within Lane Boundaries

In this task, a lane with predefined boundaries was simulated, and the car was manually controlled using a joystick. The initial position of the car was set in the center of the lane.

To validate the results:

- A **screenshot** of the simulation environment was taken, Figure 2.
- The car **lateral position** relative to the limits of the lane was plotted over time, Figure 3.
- The **joystick inputs** used to control the car were also plotted over time, Figure 3.

Figure 2 shows the simulation environment, where the car remained inside the lane during the experiment.

Figure 3 shows that the position of the car remained within the lane limits (red and green dashed lines) throughout the simulation. The center position of the car (blue line) remained within the limits, showing the effectiveness of manual control. The lower subplot shows the joystick inputs applied over time. These inputs correspond to the manual adjustments made by the user to keep the car within the lane, confirming that the car can be effectively controlled using the joystick, staying within the lane limits as required by Task 3.

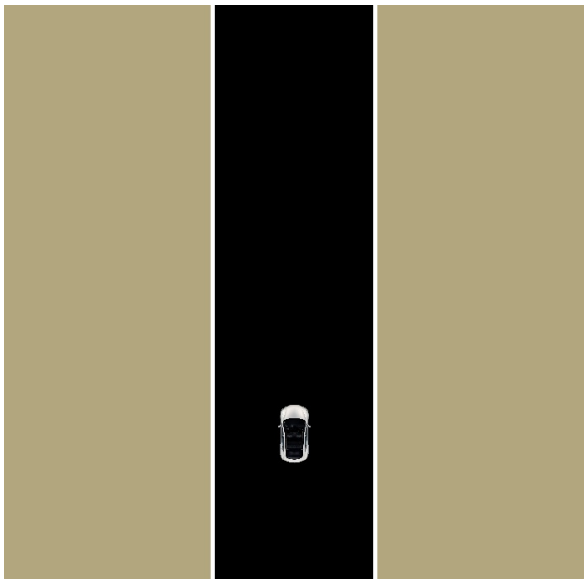


Figure 2: Simulation environment showing the car within the lane.

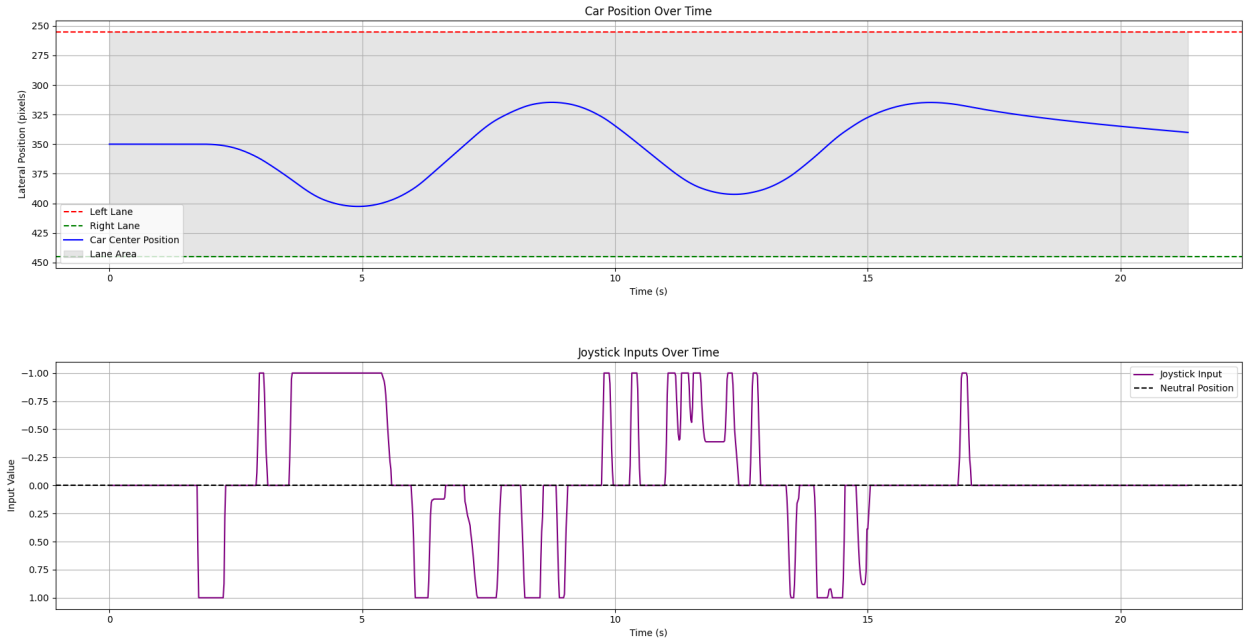


Figure 3: The upper subplot shows the car position over time, while the lower subplot shows the joystick inputs used for manual control.

3.4 Task 4 - Lane Detection

The detection of the left and right white lines of the lane was implemented as a key component of the LTA system. To accurately determine whether the car detects these lines, it was necessary to calculate the coordinates of the car's corners, mainly the coordinates of the front corners of the car as these are essential for detecting whether there was a collision.

3.4.1 Car corners coordinates

To compute the corners, we used the following equations:

Front Axle Center Calculation

Similarly, the front axle center is computed by moving half the car's length forward along its orientation θ:

$$\text{Front}_x = x + \frac{\text{Car Length}}{2} \cdot \cos(\theta)$$

$$\text{Front}_y = y + \frac{\text{Car Length}}{2} \cdot \sin(\theta)$$

where:
- x and y define the center position of the car,
- θ represents the orientation (heading angle) of the car.

Front Corner Calculations

The coordinates of the front corners of the car can be derived from the front axle center $(\text{Front}_x, \text{Front}_y)$ and the car's width.

Front Left Corner Calculation

For the front left corner, the coordinates are determined by moving half the car's width in a direction perpendicular to the car's orientation θ. This is calculated as:

$$\text{Front Left Corner}_x = \text{Front}_x + \frac{\text{Car Width}}{2} \cdot \sin(\theta)$$

$$\text{Front Left Corner}_y = \text{Front}_y - \frac{\text{Car Width}}{2} \cdot \cos(\theta)$$

Front Right Corner Calculation

Similarly, for the front right corner, the coordinates are calculated by moving half the car's width in the opposite direction perpendicular to the car's orientation:

$$\text{Front right Corner}_x = \text{Front}_x - \frac{\text{Car Width}}{2} \cdot \sin(\theta)$$

$$\text{Front right Corner}_y = \text{Front}_y + \frac{\text{Car Width}}{2} \cdot \cos(\theta)$$

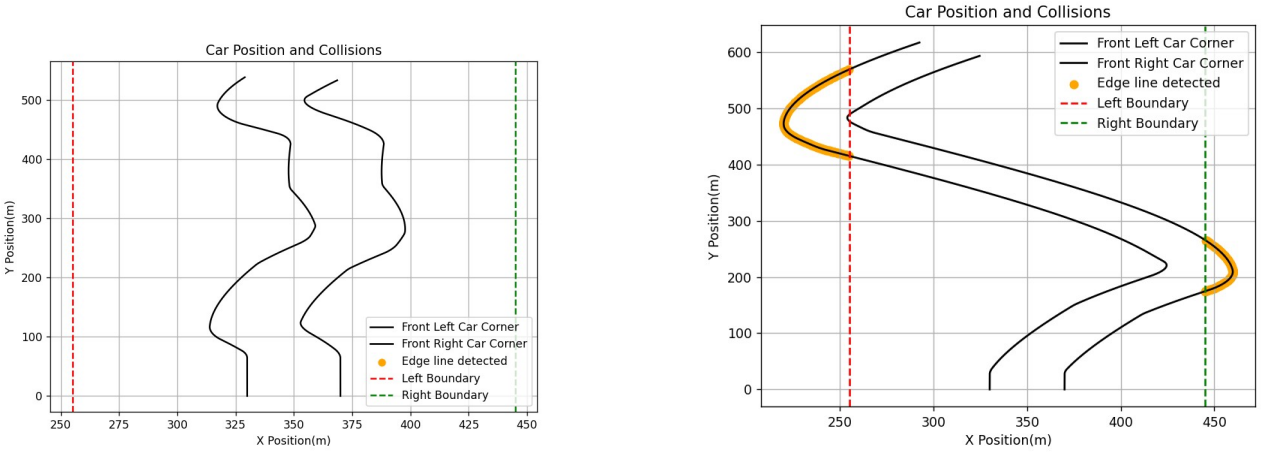
In these equations, the sine and cosine terms adjust the position of the corners based on the orientation θ of the car, ensuring they are correctly placed relative to the front axle center. These equations are derived from the fact that the initial orientation θ is $-\frac{\pi}{2}$, which aligns the car's orientation along the negative y-axis, affecting the direction of the perpendicular components.

3.4.2 Car Detection plots

At every simulation iteration, the front-left corner and front-right corner coordinates were compared to the road boundaries with a threshold of one unit added to enhance safety and avoid potential collisions. Whenever the x-coordinate of the front-left or front-right corner, including the threshold, crossed the left or right lane line, respectively, a collision detection was logged. When this threshold was exceeded, the Lane-Tracking Assist (LTA) system was activated to prevent further boundary violations.

To visualize the detections of the lane, we create two types of plots:

- **2D Trajectory Plot:** This plot shows the x and y coordinates of the car's front corners over time, highlighting the collisions with markers. It provides a spatial representation of the car's movement and its interactions with the lane boundaries.

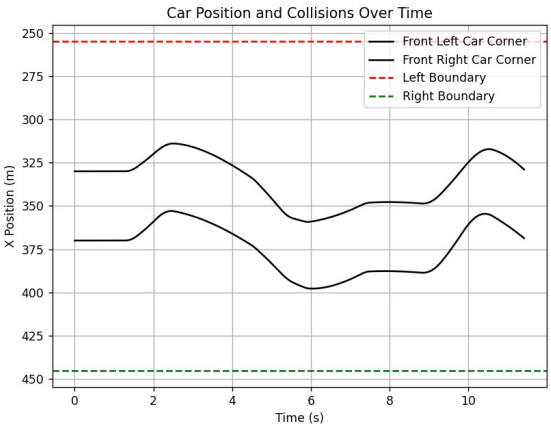


(a) X and Y coordinates without detection.

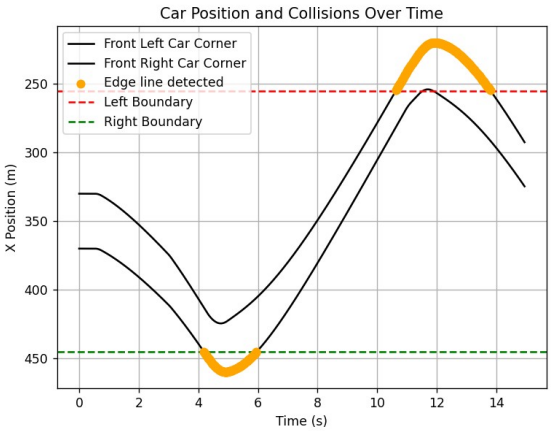
(b) X and Y coordinates with detection.

Figure 4: X and Y coordinates of front car corners with and without detection.

- **Time-Based Plot:** This plot shows the x-coordinates of the front corners as a function of time. It helps visualize when and where the collisions occurred, offering a temporal perspective of the boundary crossings.



(a) X coordinate over time without detection.



(b) X coordinate over time with detection.

Figure 5: X coordinate of front car corners over time with and without detection.

These plots clearly demonstrate the successful detection of the left and right lane boundaries over time.

3.5 Task 5: Development of a Lane-Trace Assist (LTA) Controller

For Task 5, a Lane-Trace Assist (LTA) controller was developed to prevent the vehicle from crossing the edge lines of the road. The system intervenes by adjusting the steering angle (ϕ) when the car is approaching an edge line, redirecting it back toward the center of the lane. The controller ensures smooth steering adjustments, driven by well-optimized PID parameters, to avoid abrupt changes or oscillations.

3.5.1 Lateral Error Detection

The lateral error (ε) is calculated based on the position of the car's front wheels relative to the lane boundaries. It is defined as:

$$\varepsilon = \begin{cases} x_{\text{inside\_left\_boundary}} - x_{\text{front\_left}}, & \text{if } x_{\text{front\_left}} < x_{\text{inside\_left\_boundary}} \\ x_{\text{front\_right}} - x_{\text{inside\_right\_boundary}}, & \text{if } x_{\text{front\_right}} > x_{\text{inside\_right\_boundary}} \\ 0, & \text{otherwise.} \end{cases}$$

Where:

- $x_{\text{front\_left}}$ and $x_{\text{front\_right}}$ are the x-coordinates of the car's front-left and front-right corners, respectively. These are calculated as:

$$x_{\text{front\_left}} = \text{Front}_x + \frac{\text{Car Width}}{2} \cdot \sin(\theta)$$

$$x_{\text{front\_right}} = \text{Front}_x - \frac{\text{Car Width}}{2} \cdot \sin(\theta)$$

  Front_x is the x-coordinate of the car's front axle center, calculated as:

$$\text{Front}_x = x + \frac{\text{Car Length}}{2} \cdot \cos(\theta)$$

  Here, x is the x-coordinate of the car's center and θ is the car's orientation angle.

- $x_{\text{inside\_left\_boundary}}$ and $x_{\text{inside\_right\_boundary}}$ are the inner lane boundaries, accounting for the width of the lane lines.

(a) Lateral error calculation for the left boundary.

(b) Lateral error calculation for the right boundary.

Figure 6: Lateral error calculation for the left and right boundaries.

3.5.2 PID Parameter Optimization

To ensure the effectiveness of the PID controller, an optimization process was implemented to determine the best values for the parameters (K_p, K_i, K_d).

Perturbation Strategy: The car was intentionally placed on one of the edge lines, simulating a scenario where the system needs to correct a significant deviation. This phase of simulation lasted for 10 seconds and provided critical data on how different PID configurations performed under such conditions.

After completing the optimization phase, the car was repositioned at the center of the lane to conduct a normal simulation. In this phase, the user tested the LTA controller by intentionally steering the car closer to the lane boundaries to observe if the PID controller effectively corrected the trajectory.

Optimization Process: The following functions were employed to optimize the PID parameters:

- `calculate_lateral_error(state)`: Calculates the lateral error based on the car's front-wheel positions relative to the lane boundaries.
- `compute_pid_correction(Kp, Ki, Kd, error, dt)`: Computes the steering correction based on the PID formula.
- `pid_cost_function(params)`: Evaluates the performance of a given set of PID parameters by summing the absolute lateral errors during a 10-second simulation.

- `optimize_pid()`: Uses a numerical optimization method to find the best parameters that minimize the cost function.

Simulation Reset: The simulation is reset before each optimization run to ensure consistency. This involves clearing accumulated errors, resetting the car's position, and reinitializing the PID controller.

3.5.3 PID Control

The LTA applies a PID control mechanism to compute steering corrections based on the detected lateral error. This involves:

Proportional Term (P): Provides immediate correction proportional to the magnitude of the error.

Integral Term (I): Addresses accumulated error over time to mitigate biases.

Derivative Term (D): Anticipates future deviations by considering the rate of change of the error.

The steering correction (ϕ) is computed as:

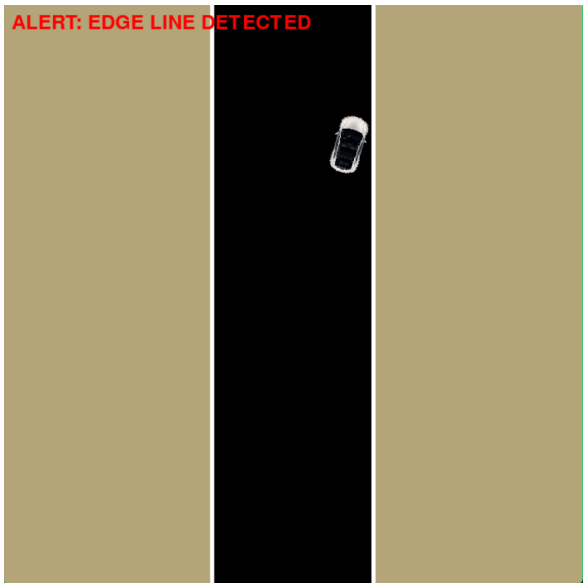
$$\phi = K_p \varepsilon + K_i \int \varepsilon \, dt + K_d \frac{d\varepsilon}{dt}.$$

The smooth steering behavior observed is a result of well-optimized PID parameters, balancing the proportional, integral, and derivative contributions.

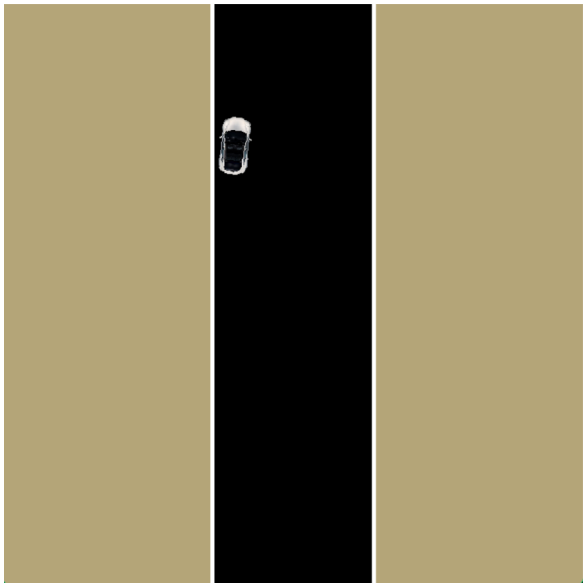
3.5.4 LTA Efectiveness

To demonstrate the effectiveness of the Lane-Trace Assist (LTA) controller, a series of images illustrating the system in operation are presented:

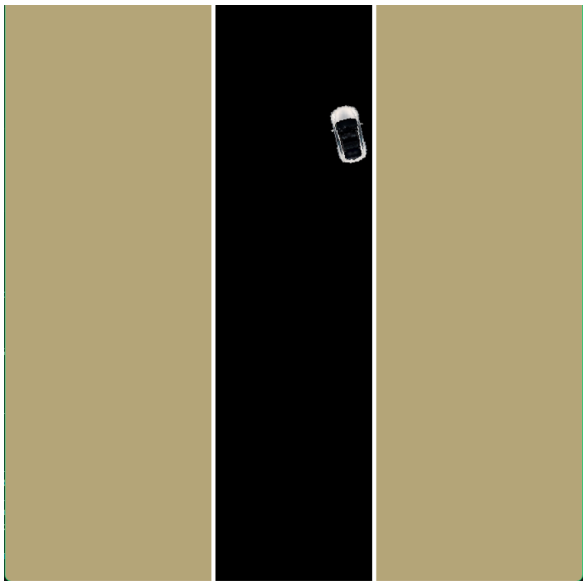
(a) left boundary detection



(b) Right boundary detection



(c) Left boundary correction



(d) Right boundary correction

Figure 7: Lane-Trace Assist (LTA) controller in action.

Further validation of the system is provided in Task 7, which includes graphical representations of the controller's behavior. These results quantitatively illustrate the system's ability to maintain lane integrity, showing:

- The evolution of the lateral error over time.

- The control inputs applied by the joystick and PID controller.
- The car's trajectory relative to the lane boundaries.

Together, the images and the graphs reinforce the effectiveness of the LTA controller in ensuring safe and stable lane-keeping performance.

3.6 Task 6 - Mathematical Demonstration of LTA System Effectiveness

To evaluate the effectiveness of the proposed Lane Tracking Assist (LTA) system, a stability analysis was conducted using Lyapunov's stability method and numerical simulations.

3.6.1 Lyapunov Stability Analysis

The Lyapunov candidate function chosen for this analysis is:

$$V(e) = \frac{1}{2}e^2,$$

where e represents the lateral error, defined as the deviation of the car from the center of the lane. This quadratic Lyapunov function was selected because it is positive definite ($V(e) > 0$ for $e \neq 0$, and $V(e) = 0$ at $e = 0$), radially unbounded, and simple to analyze.

The time derivative of $V(e)$ is given by:

$$\dot{V}(e) = \frac{\partial V}{\partial e} \cdot \dot{e} = e \cdot \dot{e}.$$

Using the error dynamics derived from the LTA system's PID controller:

$$\dot{e} = -K_p \cdot e - K_d \cdot \dot{e},$$

where $K_p = 1.0$ and $K_d = 0.06$, the derivative of the Lyapunov function becomes:

$$\dot{V}(e) = e \cdot (-K_p \cdot e - K_d \cdot \dot{e}).$$

Substituting the gain values:

$$\dot{V}(e) = -1.0 \cdot e^2 - 0.06 \cdot e \cdot \dot{e}.$$

The term $-1.0 \cdot e^2$ ensures that $\dot{V}(e) \leq 0$, indicating that the system dissipates energy over time. This confirms the system's asymptotic stability, provided that $K_p > 0$ and $K_d > 0$. The alignment of the Lyapunov analysis with the PID controller is deliberate, as the stability properties of the LTA system are governed by the same proportional (K_p) and derivative (K_d) gains used in the PID algorithm. This ensures consistency between the theoretical stability criteria and the practical control implementation.

3.6.2 Simulation Results

Numerical simulations validated the theoretical stability analysis. The system was tested over a 40-second duration, starting with an initial lateral error of 50 pixels. Key performance metrics include:

- **Maximum Error:** 122.35 pixels.
- **Mean Error:** 17.12 pixels.
- **Average Response Time:** 0.0356 seconds.

Figures 9 and 8 illustrate the system's performance:

- The lateral error decreases monotonically to zero, demonstrating the controller's ability to correct deviations effectively.
- The Lyapunov function $V(e)$ decreases over time, confirming energy dissipation and system stability.

The choice of K_p and K_d in the simulations mirrors the values used in the PID controller of the LTA system. This alignment is crucial as it ensures that the numerical results directly validate the theoretical analysis. The agreement between theory and simulation indicates that the system effectively reduces lateral deviations while maintaining stability.

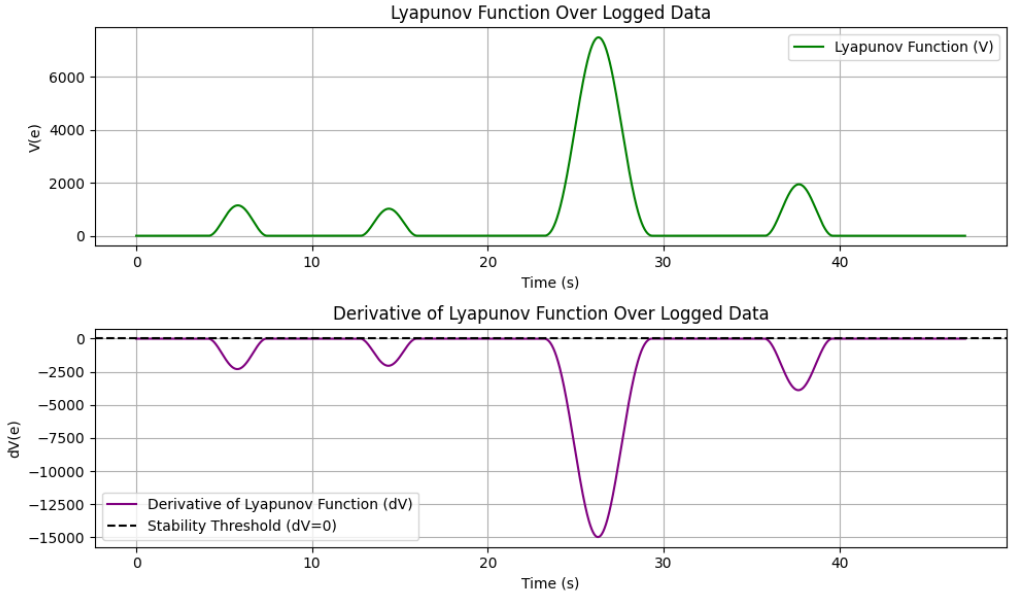


Figure 8: Lyapunov function and its derivative over time.

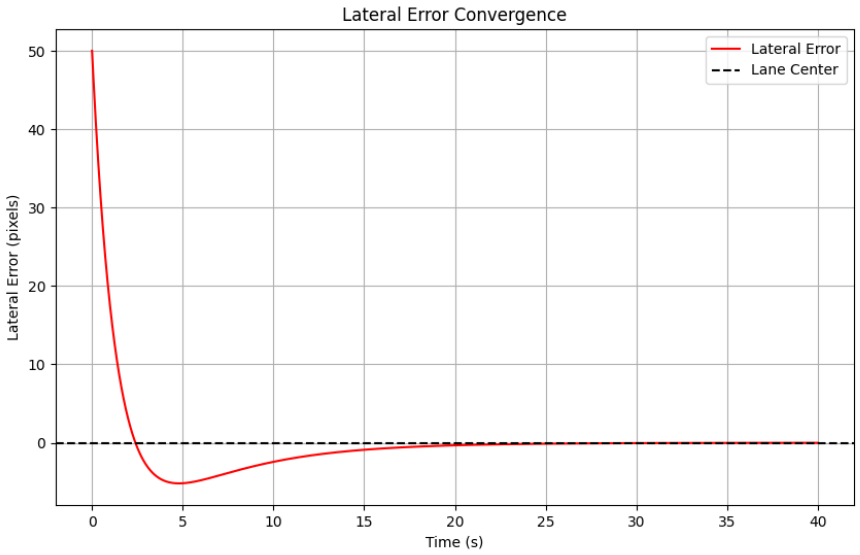


Figure 9: Lateral error convergence.

3.6.3 Conclusion

The use of proportional (K_p) and derivative (K_d) gains from the LTA system's PID controller ensures that both the theoretical analysis and practical implementation rely on the same control parameters. This consistency guarantees that the stability criteria demonstrated through Lyapunov analysis are directly applicable to the LTA system. The simulation results confirm that the controller reduces lateral deviations effectively and stabilizes the vehicle in the lane center. The absence of oscillations and overshoot further validates the robustness and reliability of the LTA system.

3.7 Task 7: Validation of the LTA System

The lateral error, representing the distance of the car front corners to the edge line, was monitored throughout the simulation. The error was minimized by the LTA system using a PID controller.

Figure 10 shows the evolution of the lateral error over time. Positive and negative values represent deviations to the left and right, respectively. The LTA system successfully reduced these deviations, as shown in Figure 11, the joystick input (green) represents user-induced steering errors, while the PID correction (orange) illustrates the system response. When the user attempts to steer the car outside the lane boundaries, the PID controller applies corrective actions to counteract these deviations. This interaction demonstrates how the LTA system combines manual inputs and automated assistance to maintain lane adherence effectively.

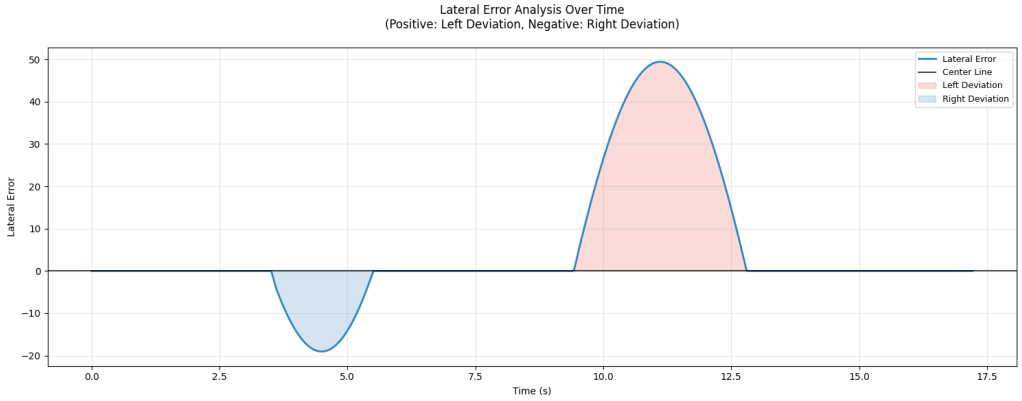


Figure 10: Lateral error over time. Positive values indicate left deviations, while negative values indicate right deviations. The LTA system effectively minimized these deviations.

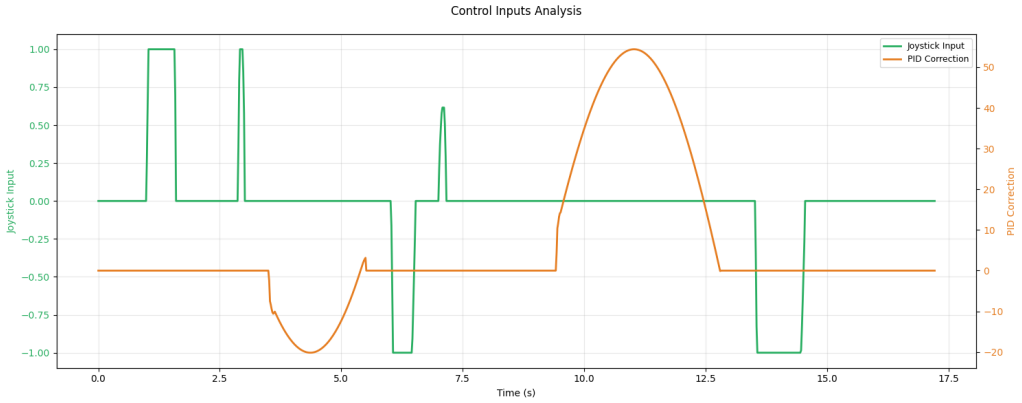


Figure 11: Control inputs over time. Joystick inputs (green) are corrected by the LTA system using PID control (orange).

The trajectory of the car is presented in Figure 12, highlighting the car movement along the lane. The system consistently kept the car within the lane boundaries, even when we forced inputs to steer it out of the lane (indicated by the red and green dashed lines). The color gradient in Figure 12 represents the simulation time, allowing for the visualization of both spatial and temporal aspects of the car movement. This highlights the consistency of the system's control throughout the entire simulation period.

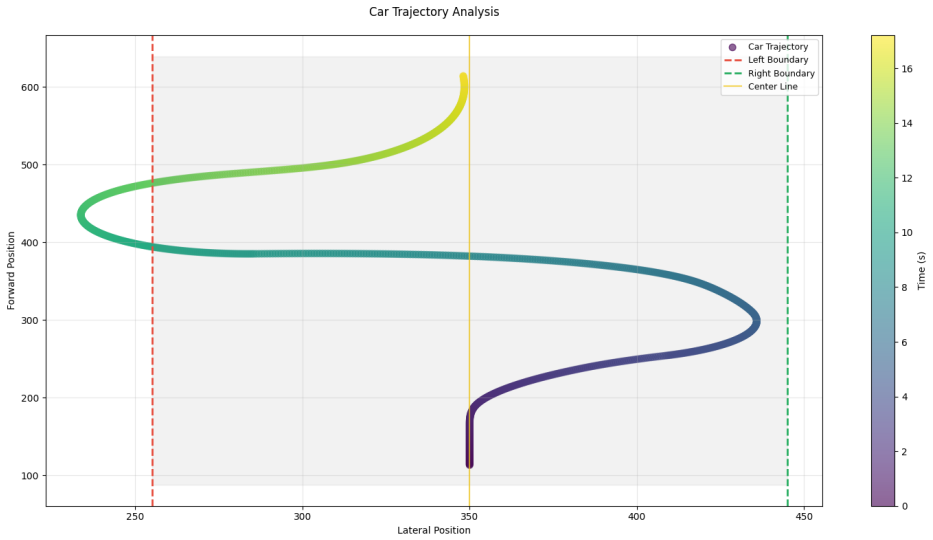


Figure 12: Car trajectory along the lane. The car remained within the boundaries throughout the simulation, with deviations corrected by the LTA system.

The following metrics were used to evaluate the system performance, providing insight into its responsiveness, effectiveness, and control dynamics:

- **Average Response Time:** 2.69 seconds - the average time taken by the system to detect and correct a deviation from the lane center.
- **Number of Corrections:** 2 interventions - the total number of successful corrections made when the car approached the lane boundaries during this simulation.
- **Correction Success Rate:** 100% - all detected deviations were successfully corrected by the system during this simulation.
- **Maximum PID Correction:** 54.43 rad/s - the highest steering rate correction applied by the PID controller to maintain the car within the lane boundaries.
- **Mean PID Correction:** 8.83 rad/s - indicating that most steering corrections were moderate and well-controlled.
- **Maximum Joystick Input:** 1.00 (normalized) - the largest normalized steering input provided by the user during the simulation, where -1.0 represents a maximum left turn and +1.0 a maximum right turn.

4 Conclusion

The development and evaluation of the Lane-Tracking Assist (LTA) system provided a comprehensive demonstration of the integration of kinematics, control theory, and real-time simulations. The tasks addressed key components such as manual control, lane detection, PID-based steering corrections, and Lyapunov stability analysis. Through kinematic and control models we were able to successfully simulated realistic lane-keeping behavior, with the LTA

system correcting deviations and maintaining stability within lane boundaries. Through the use of PID control, we ensured that the system achieved smooth steering corrections, minimizing oscillations and overshoot. The optimization of the K_p, K_i, and K_d gains proved essential in balancing responsiveness and stability, with performance validated through extensive simulation tests. The Lyapunov stability analysis mathematically confirmed the system's asymptotic stability Simulation results further validated the system's performance, with metrics such as the maximum lateral error, mean error, and response time illustrating the effectiveness of the LTA controller and the graphical analysis of the car's trajectory, lateral error, and control inputs confirmed that the system maintained vehicle stability under both nominal and perturbed conditions. In conclusion, the LTA system effectively achieved its goal of lane-keeping through a combination of mathematical rigor and practical implementation.